



KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Compiler Project Proposal Report Manual

Tools: Using Flex & Bison

Submitted To:

Md. Badiuzzaman Shuvo

Lecturer

Department of Department of Computer
Science and Engineering (CSE)

Khulna University of Engineering &
Technology (KUET)

Subah Nawar

Lecturer

Department of Department of Computer
Science and Engineering (CSE)

Khulna University of Engineering &
Technology (KUET)

Submitted By:

Tawhidul Hasan

Roll No: 2107004

Year: Third

Semester: Second

Department of Computer Science and
Engineering (CSE)

Khulna University of Engineering &
Technology (KUET)

Date of Submission: January 12, 2026

Objectives:

- To design a custom high-level programming language using a wizard/magic theme
- To implement a lexical analyzer using Flex for token generation
- To implement a syntax analyzer using Bison for grammar parsing
- To map wizard-themed keywords and symbols **to** equivalent C language constructs

Introduction:

Compiler design is a fundamental area of computer science that focuses on translating high-level programming languages into machine-understandable code. This project proposes the design and implementation of a Wizard-Themed Programming Language, inspired by magical spellcasting concepts, using Flex for lexical analysis and Bison for syntax analysis.

The proposed language aims to make programming more engaging, readable, and creative by replacing traditional programming keywords with wizard-inspired terminology such as spells, potions, incantations, and enchantments. Internally, the language constructs will be mapped to equivalent C language constructs, enabling execution through semantic actions.

Flex (Fast Lexical Analyzer)

Flex is a tool used to generate a lexical analyzer. It reads the source program and converts character streams into meaningful tokens such as keywords, identifiers, operators, and literals. In this project, Flex will recognize wizard-themed keywords and symbols.

Bison (Parser Generator)

Bison is a parser generator used to analyze the syntax structure of the program. It checks whether the token sequence produced by Flex follows the defined grammar rules. Bison will be used to validate control structures, expressions, and function definitions of the wizard language.

SL NO	Wizard Keyword/Symbol	String/Keyword/Symbol in C	Description
1	ENTER_HOGWARTS	main() {	Program entry point
2	EXIT_HOGWARTS	}	Program closure
3	HOUSE	{	Block open
4	ENDHOUSE	}	Block close
5	Gryffindor	Declaration Block	House role for declarations
6	Slytherin	Function Block	House role for functions
7	Hufflepuff	Condition Block	House role for conditions
8	Ravenclaw	Loop Block	House role for loops
9	INT	int	Integer data type
10	FLOAT	float	Float data type
11	DOUBLE	double	Double data type
12	LONG	long	Long integer data type
13	CHAR	char	Character data type
14	BOOL	bool	Boolean data type
15	VOID	void	No return type
16	DECLARE	type variable =	Declaration statement form
17	AS	type binding keyword	Type binding keyword
18	=	=	Assignment operator
19	:=	initialization assignment	Initialization assignment
20	IF	if	If keyword
21	FI	}	End of if block
22	ELSE	else	Else keyword
23	CHECK	if (condition)	Conditional check form
24	THEN	{	Start condition block
25	ENDCHECK	}	End condition block
26	LOOP	while	Loop keyword

SL NO	Wizard Keyword/Symbol	String/Keyword/Symbol in C	Description
27	DO	{	Start loop block
28	ENDLOOP	}	End loop block
29	BREAK	break	Break statement
30	CONTINUE	continue	Continue statement
31	SPELL	function definition	Function definition keyword
32	ENDSPELL	}	End function block
33	SUMMON	function call	Function call keyword
34	WITH	argument passing	Argument passing keyword
35	RETURN	return	Return statement
36	CAST	printf()	Output statement
37	PROPHECY	printf (formatted output)	Formatted output
38	INPUT	scanf()	Input statement
39	POTION	{	Scope block start
40	ENDPOTION	}	Scope block end
41	SORTING_HAT	semantic_check()	Semantic validation entry
42	ENDHAT	end semantic block	End semantic block
43	CHECK_TYPES	type checking	Type validation
44	CHECK_DECLARATIONS	symbol table validation	Declaration validation
45	+	+	Addition operator
46	-	-	Subtraction operator
47	*	*	Multiplication operator
48	/	/	Division operator
49	%	%	Modulus operator
50	AND	&&	Logical AND
51	OR		Logical OR
52	NOT	!	Logical NOT

SL NO	Wizard Keyword/Symbol	String/Keyword/Symbol in C	Description
53	XOR	^	Logical XOR
54	POWO	pow()	Exponentiation function
55	RADIX	sqrt()	Square root function
56	FLOORUS	floor()	Floor function
57	CEILUS	ceil()	Ceiling function
58	ABSOLUTUS	abs()	Absolute value function
59	LOGUS	log()	Logarithm function
60	SINUS	sin()	Sine function
61	COSINUS	cos()	Cosine function
62	TANUS	tan()	Tangent function
63	IDENTIFIER	[a-zA-Z_][a-zA-Z0-9_]*	Identifier token pattern
64	NUMBER	[0-9]+	Number token pattern